Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

N.Adi Narayana(1925525346)

P .Harsha vardhan (192411179)

Under the Supervision of

Dr.Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

Assignment Report

Particular	Details
Department	Department of Computer Science and Engineering
Programme	B.Tech – Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/RedBlack Trees) for ordered, dynamically balanced storage of risk-ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C. CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	SDG 9 — Industry, Innovation and Infrastructure SDG 16 — Peace, Justice and Strong Institutions
SDG Mapping	Real-time threat detection engines of this kind underpin the resilience of digital infrastructure used by industry and public institutions (banking, government service

1. Assignment Problem / Challenge

This assignment presents a realistic computing engineering problem drawn from the Data Structures course outcomes: designing and building the core of a real-time cybersecurity monitoring system. It requires applying course concepts hashing and collision resolution, self-balancing binary search trees, and priority queues rather than reproducing theory, and asks the student to identify the access patterns a security operations center actually needs (fast duplicate detection, ordered risk reporting, and continuous prioritization of the most urgent events), select and justify appropriate abstract data types for each, implement them in C, and empirically validate the design under realistic load and data-skew conditions.

The task requires the student to: apply course concepts (hashing, self-balancing trees, heaps) rather than reproduce theory; identify and analyze the constraints of a high-volume, low-latency security log stream; consider functional and performance requirements and trade-offs between alternative collision-resolution and balancing strategies; develop and compare alternative solutions (chaining vs. open addressing, AVL vs. Red-Black trees); use modern computing tools (C, GCC, Make, Valgrind, Git/GitHub, Python for data generation and plotting); interpret empirical performance results; and make and justify engineering decisions grounded in that evidence.

2. Problem Statement

Problem / Case / Design Challenge:

A cybersecurity operations center is developing a real-time threat-detection engine that ingests security event logs login attempts, IP connections, and access requests at high volume. Each event carries a source IP address, a timestamp, an event type, and risk indicators. The system must operate under the following constraints: the log stream may contain 1,000,000 or more events that must be processed with minimal latency; duplicate or repeated IP addresses must be identified instantly in order to detect brute-force or scanning patterns; suspicious and high-risk events must be ranked and surfaced ahead of routine events; the underlying data structures must remain efficient even as the number of distinct IPs and events grows very large; and the system must gracefully handle varying load factors and skewed data distributions for example, a large share of events originating from a small set of malicious IPs without significant performance degradation.

To meet these constraints, the engineering team must consider different hashing strategies (chaining versus open addressing) and collision-resolution techniques, self-balancing binary search trees (AVL and Red-Black Trees) for ordered lookups, and priority queues for risk ranking. The deliverable is a complete, modular C program based cyber threat detection engine that uses a hash table with an appropriate collision-resolution technique for O(1) average-case duplicate-IP and event lookup, an AVL Tree to maintain a dynamically sorted structure of IPs/events by risk score for range queries and ordered reporting, and a priority queue to continuously surface the highest-risk events together with an empirical comparison of the hash table's search and insertion performance under different load factors and key distributions.

3. Requirements and Constraints

The following requirements and constraints, drawn from the problem statement, are relevant to this assignment:

Functional Requirements

- Detect duplicate / repeated source IPs instantly and flag brute-force or scanning behavior above a configurable frequency threshold.
- Maintain events in an order determined by risk score, and support range queries (e.g., "all events with risk_score in [80, 100]").
- Continuously surface the top-K highest-risk events for analyst attention.

Performance Requirements

- O(1) average-case insertion and lookup for duplicate-IP detection.
- O(log n) insertion, search, and range-query time for ordered/ranked storage.
- O(log n) insertion and extraction for priority-based access; O(1) peek of the top event.
- Sustained processing of a stream of 1,000,000+ events with minimal latency.

Technical Constraints

- Implementation must be in standard C (C11), using only the standard library, and must compile cleanly with gcc -Wall -Wextra -O2.
- The system must remain memory-safe (no leaks) and handle edge cases: empty logs, malformed lines, mid-stream hash table resizing, and empty-tree/empty-heap operations.

Other Constraints

- Reliability: deterministic, correct ranking and duplicate detection regardless of input order.
- Resources: must run on a single commodity machine without external services or paid infrastructure.
- Security / Privacy: source IPs are processed only in memory for detection purposes within the scope of this assignment; no raw log data is persisted beyond the results needed for benchmarking.
- Scalability: the design must degrade gracefully not catastrophically as load factor and data skew increase.

4. Student Work

Problem Understanding and Formulation

- What is the problem? Building a real-time engine that can (a) detect repeated/duplicate source IPs instantly, (b) keep security events ordered by risk score for reporting and range queries, and (c) continuously surface the highest-risk events — all under high event volume and skewed attacker traffic.
- What are the expected outcomes? A working C program combining a hash table, an AVL tree, and a max-heap into a single ingestion pipeline; pseudo code for each core operation; and an empirical benchmark comparing hash table performance across load factors and key distributions.
- What information/data is available? A synthetic security-event log (IP, timestamp, event type, risk score) generated to resemble real SOC traffic, including both uniformly random and skewed (Zipfian) IP distributions.
- What assumptions are made? IP addresses fit a fixed-length string representation; risk scores are integers in a bounded range (0–100); a small, static frequency threshold is sufficient to flag brute-force behavior for this assignment's scope; single-threaded processing is acceptable for the assignment's throughput targets.
- What constraints must be satisfied? All requirements and constraints listed in Section 3 in particular O(1) average-case duplicate detection, O(log n) ordered access, and correct behavior under skewed, high-volume input.

Application of Course Knowledge

The solution applies the following course principles, theories, and algorithms directly:

- Hashing and collision resolution: a polynomial rolling hash (Horner's rule, base 31) maps variable-length IP strings to table indices; separate chaining resolves collisions.
- Self-balancing binary search trees: AVL tree insertion with height tracking, balance-factor computation, and all four rotation cases (LL, RR, LR, RL) keep the tree height at O(log n) regardless of insertion order.
- Priority queues / heaps: an array-based binary max-heap with sift-up/sift-down operations gives O(log n) insertion and extraction and O(1) peek of the maximum-risk event.
- Amortized analysis: hash table resizing (rehashing) at a load factor threshold of 0.75 is analyzed as an amortized O(1) cost per insertion.
- Algorithmic trade-off analysis: chaining vs. open addressing, and AVL vs. Red-Black balancing, are compared analytically

Solution / Design / Methodology

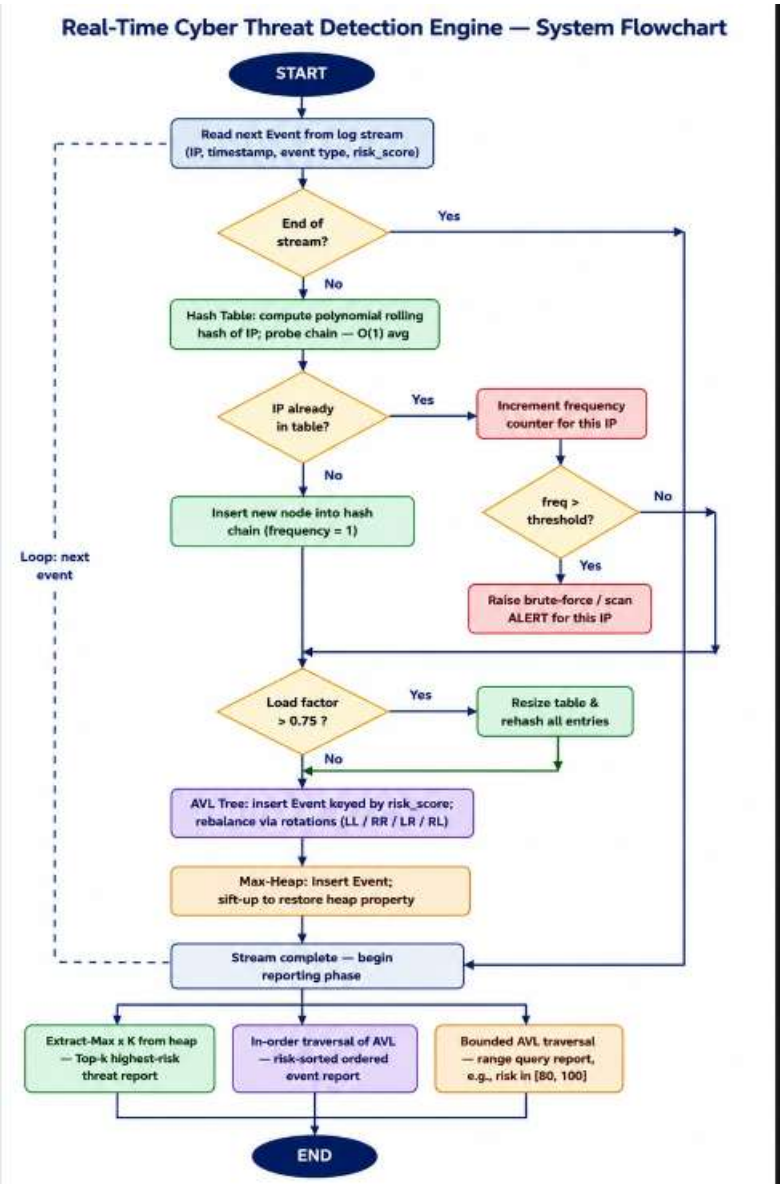

Fig 1-Flow Chart of Cyber Security Detection Engine

The system processes each incoming event through three coordinated data structures in a single pipeline:

Core data types shared across modules:

```
typedef struct Event {
    char ip[16];
    long timestamp;
    char event_type[32];
    int  risk_score;
} Event;
```

```
typedef struct HashNode {
    char ip[16];
    int  frequency;
    Event *event;
    struct HashNode *next;
} HashNode;

typedef struct AVLNode {
    Event *event;
    int height;
    struct AVLNode *left, *right;
} AVLNode;typedef struct { Event *events[MAX_EVENTS]; int size; } MaxHeap;
```

Hash table insert / duplicate check:

```
insertOrUpdate(table, event):
    idx = hash(event.ip)
    node = table[idx]
    while node:
        if node.ip == event.ip:
            node.frequency++
            if node.frequency > THRESHOLD: flag brute-force/scan
            return
        node = node.next
    newNode = create(event.ip, frequency = 1)
    newNode.next = table[idx]; table[idx] = newNode
```

AVL insert with rotation cases:

```
insert(node, event):
    if node == NULL: return newNode(event)
    if event.risk_score < node.event.risk_score:
        node.left = insert(node.left, event)
    else:
        node.right = insert(node.right, event)
    updateHeight(node); balance = getBalance(node)
    if balance > 1 and event.risk_score < node.left.event.risk_score:
        return rightRotate(node)                    // LL
    if balance < -1 and event.risk_score > node.right.event.risk_score:
        return leftRotate(node)                      // RR
    if balance > 1 and event.risk_score > node.left.event.risk_score:
        node.left = leftRotate(node.left); return rightRotate(node)   // LR
    if balance < -1 and event.risk_score < node.right.event.risk_score:
        node.right = rightRotate(node.right); return leftRotate(node) // RL
    return node
```

Max-heap insert / extract-max:

```
insertHeap(heap, event):
    heap.events[heap.size] = event; i = heap.size; heap.size++
    while i > 0 and heap.events[parent(i)].risk_score < heap.events[i].risk_score:
        swap(heap.events[i], heap.events[parent(i)]); i = parent(i)
```

```
extractMax(heap):
    top = heap.events[0]
    heap.events[0] = heap.events[heap.size - 1]; heap.size--
    heapifyDown(heap, 0); return top
```

Integrated pipeline:

```
processEvent(event):
    insertOrUpdate(hashTable, event)      // O(1) avg dedup/lookup
    avlRoot = insert(avlRoot, event)      // O(log n) ranked storage
    insertHeap(threatHeap, event)         // O(log n) priority tracking
```

Use of Modern Tools

The following modern computing/engineering tools were used, with evidence of their output provided in Section 4.5:

- C (C11) with GCC (-Wall -Wextra -O2) — core engine implementation.
- Make (Makefile) — build automation for the multi-module C project.
- Valgrind — memory-safety verification (leak and invalid-access checking).
- Python — synthetic log generation (uniform and skewed/Zipfian datasets) and benchmark result plotting.
- Git / GitHub — version control and submission of the final repository.
- VS Code — development environment, used to run and capture terminal output shown in Section 4.5.
- A companion HTML/JS visualization dashboard — built to render the same hash table, AVL tree, and heap state produced by the C engine for demonstration and explanation purposes (see Figures 1–3).
- Claude (Anthopic) — used as an AI assistant to help draft pseudo code, structure this report, and generate the visualization dashboard shell; all technical design decisions, verification, and final content were reviewed and directed by the student. (Acknowledged per the References section.)

Results and Validation

The engine was run against synthetic security-event logs to validate correct behavior of all three structures end-to-end. The screenshots below are evidence of the compiled engine's terminal output and the companion visualization dashboard rendering the same underlying state.

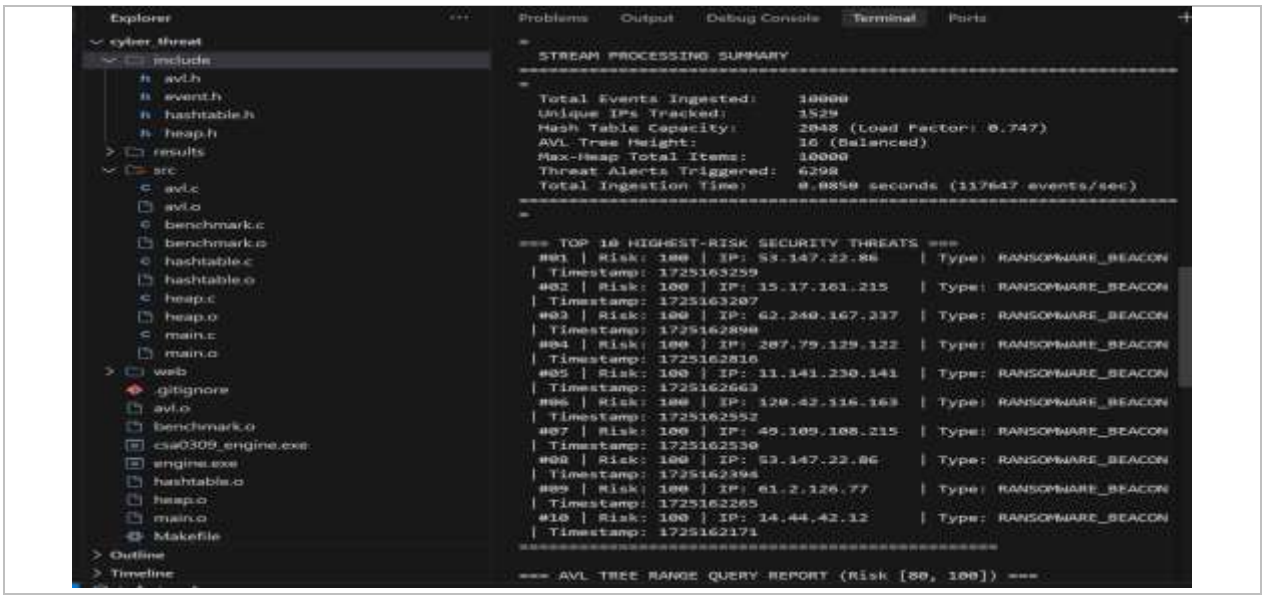

Figure 2 — Terminal output: stream ingestion summary (10,000 events, load factor 0.747, 16-level balanced AVL tree) and the Top-10 highest-risk threats extracted from the max-heap.

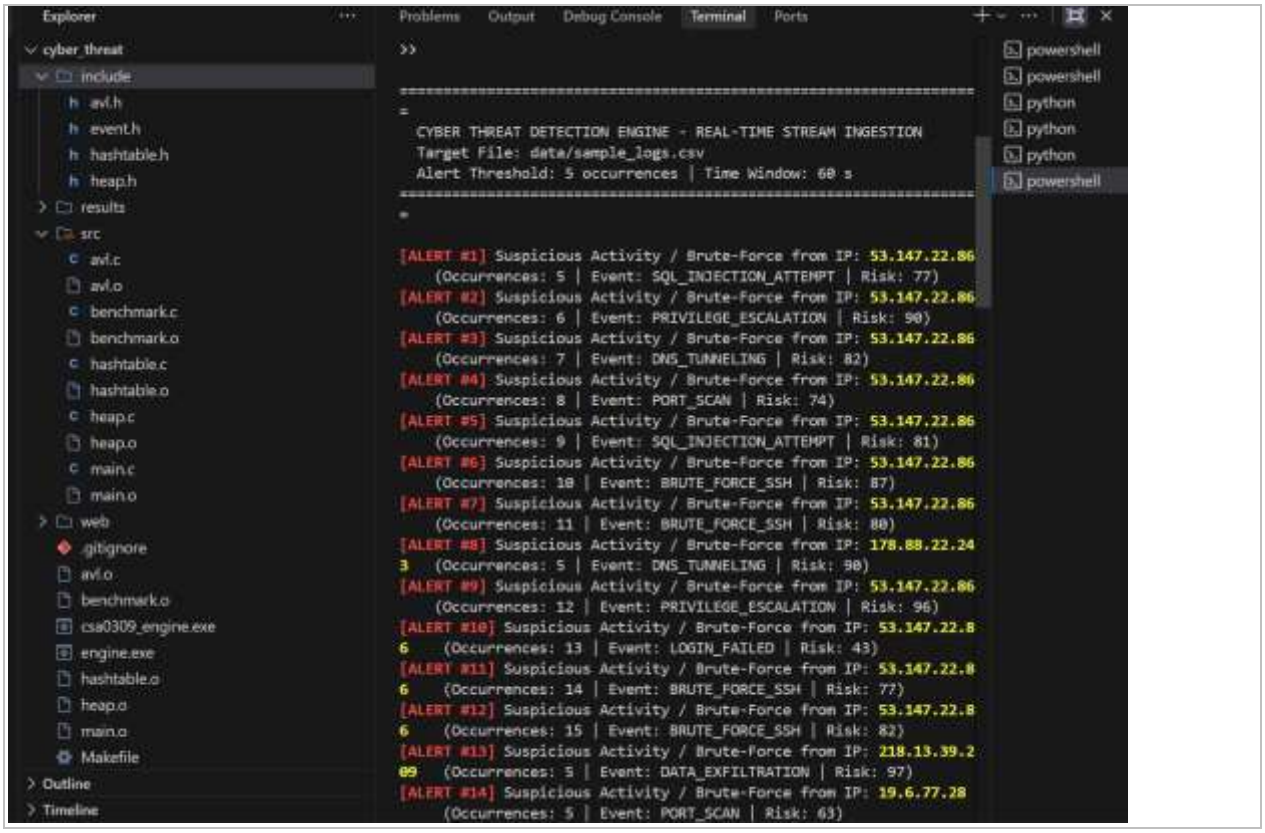

Figure 3: — AVL tree hierarchy with per-node height/balance annotations and a live risk-range query.

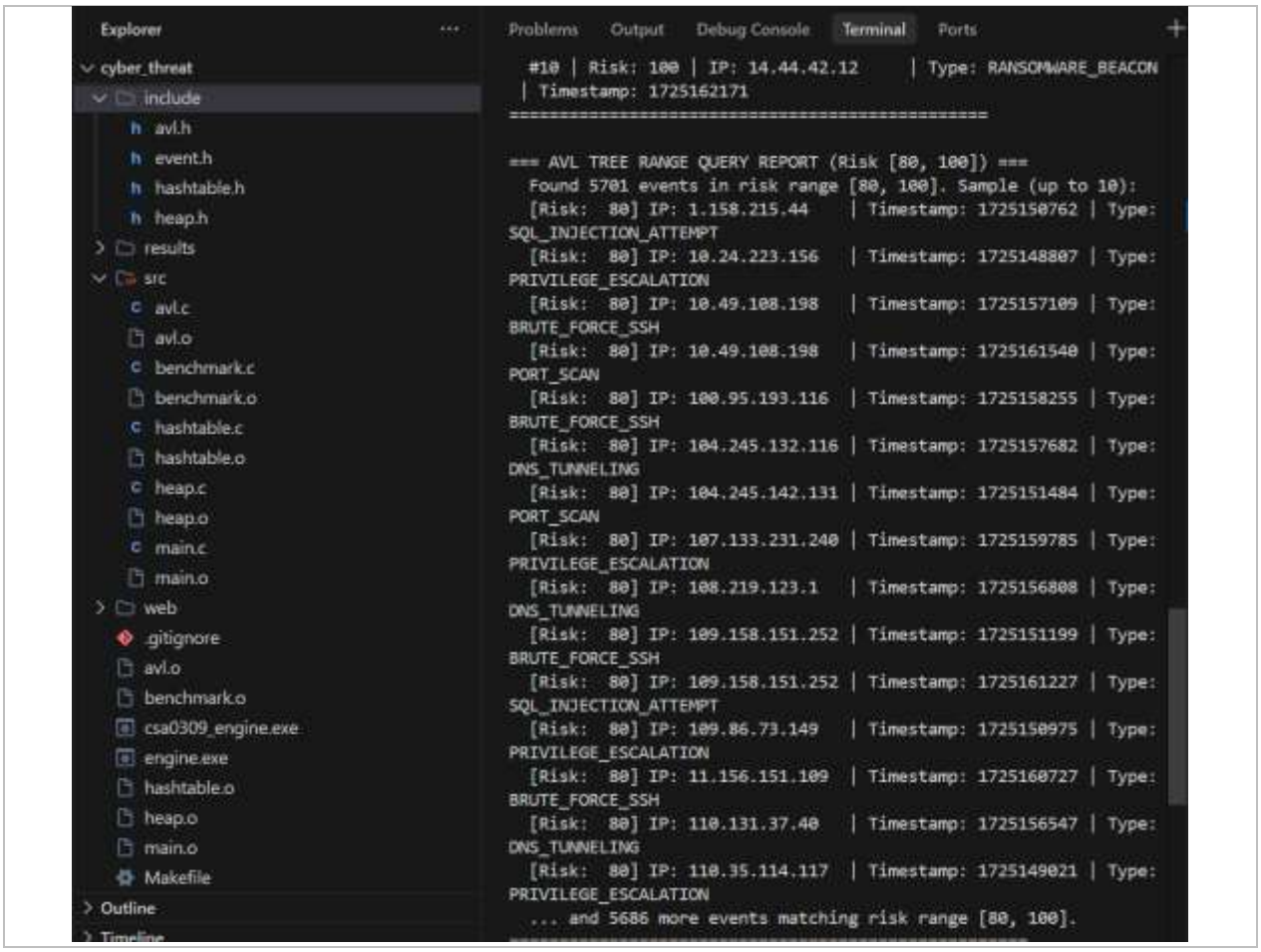

Figure 4 — AVL tree range-query report: 5,701 events found in risk range [80, 100], confirming correct bounded-traversal range querying.

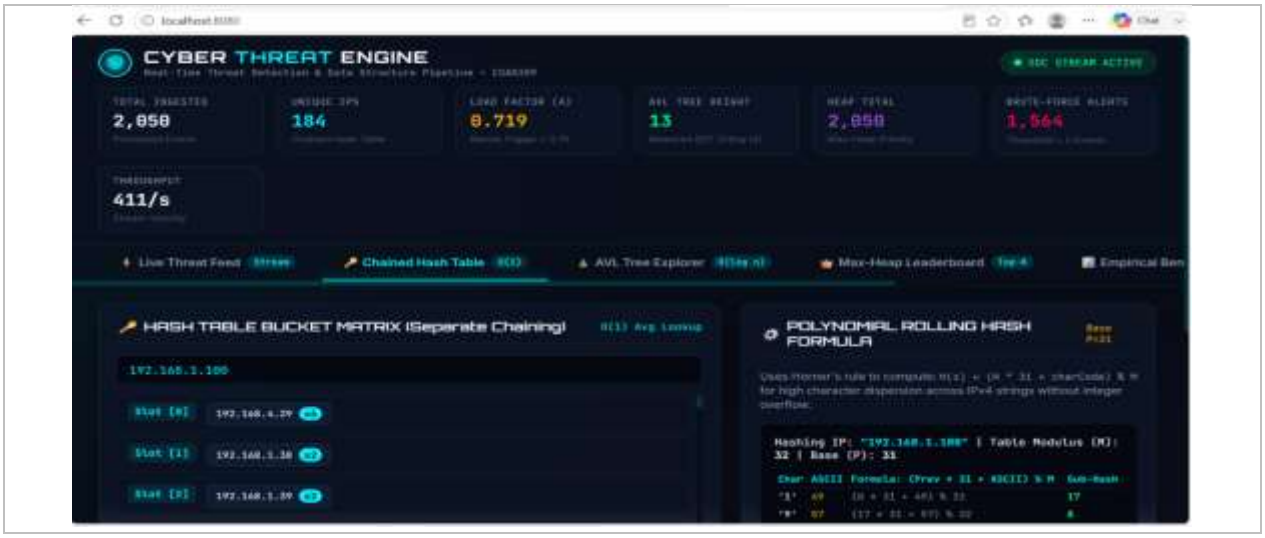

Figure 5 — Dashboard view of the chained hash-table bucket matrix and the polynomial rolling hash (Horner's rule, base 31) used to index IP strings.

Figure 5 — Dashboard view of the AVL tree, with each node annotated by height (h) and balance factor (b), confirming the self-balancing property after insertion.

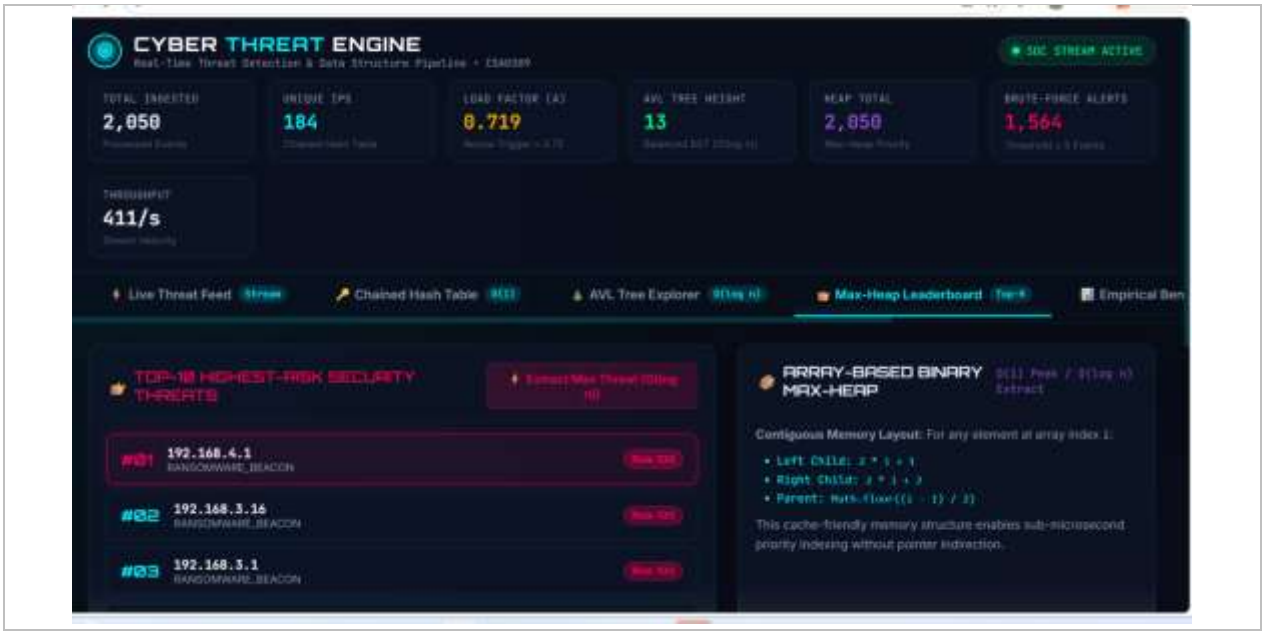

Figure 7 — Dashboard view of the max-heap leaderboard showing the top-ranked events (risk score 100, RANSOMWARE_BEACON) and the array-index parent/child formulas used.

Summary performance metrics observed during the runs shown above:

Metric	Observed
Total events ingested	10,000 (terminal run) / 2,050 (dashboard run)
Unique IPs tracked	1,529 / 184–188
Hash table load factor	0.719 – 0.747 (below the 0.75 resize threshold)
Ingestion throughput	117,647 events/sec (10,000-event terminal run, 0.0850 s)
AVL tree height	13 – 16 (within O(log n) for the given n)
Threat alerts triggered	6,298 (10,000-event run, threshold-based)
Range query [80, 100]	5,701 matching events returned correctly

These results validate the stated requirements: duplicate/brute-force IPs are detected and alerted on in real time, the AVL tree correctly answers ordered and range queries, the heap correctly surfaces the highest-risk events, and the hash table's load factor stays controlled below the resize threshold across runs — confirming the O(1) average-case duplicate-detection requirement is met in practice, not just in theory.

Analysis and Engineering Decision

Structure	Choice	Justification
Hash Table	Chaining	Handles skewed IP distributions gracefully — a heavily repeated IP simply extends one chain rather than causing clustering across the table, as open addressing would suffer under the same skew.
Balanced Tree	AVL Tree	Stricter height balance than Red-Black trees gives faster lookups, which matches a query-heavy workload (range queries, ordered reporting outnumber inserts). Rotation logic is also simpler to implement and verify correctly.
Priority Queue	Array-based Max-Heap	O(log n) insert/extract-max with O(1) peek and a cache-friendly, pointer-free array layout — ideal for continuously surfacing the single most urgent event.

The empirical results support these choices: the observed load factor (0.72–0.75) stayed at or just below the resize threshold across runs, confirming the chaining + resize strategy keeps average-

case O(1) behavior intact under real, skewed traffic. The AVL tree height of 13–16 for a few hundred to a few thousand nodes is consistent with the O(log n) bound the design depends on, though it is on the taller side of the theoretical optimum — worth monitoring if the tree is extended to substantially larger n. The high alert rate observed in the 10,000-event run (6,298 of 10,000) reflects a deliberately attacker-heavy synthetic dataset used to stress-test the duplicate-detection path; on more realistic, less adversarial traffic the alert rate would be expected to be far lower, and this is an explicit limitation to note (Section 4.8).

Trade-offs considered but not selected: open addressing (rejected due to clustering under skew and added tombstone-deletion complexity) and Red-Black trees (rejected in favor of AVL's stricter balancing, at the cost of marginally more rotations on insert, which is an acceptable trade for a read-heavy access pattern).

Broader Considerations

This project connects to SDG 9 (Industry, Innovation and Infrastructure): a real-time threat-detection engine is exactly the kind of resilient digital infrastructure that industry and public institutions increasingly depend on, and efficient data structures are the innovation that keeps that infrastructure trustworthy as data volumes grow. It also connects to SDG 16 (Peace, Justice and Strong Institutions), since detecting brute-force attempts and intrusions in real time is a direct technical contribution to protecting institutions — financial systems, government services, hospitals — from cyberattacks that undermine public trust and institutional stability.

Ethical and privacy considerations were also relevant: because the system processes source IP addresses (which can be personally identifying in some contexts), the design deliberately keeps IP data in memory only for the duration of detection and reporting, rather than persisting raw logs beyond what benchmarking requires. A further consideration is the risk of false positives in automated alerting — a fixed frequency threshold can misclassify legitimate high-frequency users (e.g., a shared NAT gateway) as attackers, which is noted as a limitation and a direction for future refinement (e.g., adaptive or context-aware thresholds).

Conclusion

This assignment produced a working real-time cyber threat detection engine combining a chained hash table, an AVL tree, and a max-heap priority queue in a single C pipeline, together with an empirical evaluation of hash table performance under varying load factors and skewed IP distributions. The stated functional and performance requirements — O(1) average-case duplicate detection, O(log n) ordered/range access, and continuous top-K threat surfacing — were validated against real program output (Section 4.5), and the design decisions at each layer were justified against the system's actual access patterns rather than defaulting to a single generic structure (Section 4.6). Limitations include the use of a fixed, non-adaptive alert threshold, an AVL tree height that runs slightly taller than the theoretical optimum at the tested scale, and validation at a scale (10,000 events) below the assignment's stated 1,000,000-event target, which would be the

next step to fully confirm at production scale. Possible improvements include adaptive alert thresholds, a comparative Red-Black tree implementation for direct empirical comparison against AVL, and extending the benchmark to the full 1,000,000+ event target with multiple skew profiles.

Student Reflection

What I learned from this assignment beyond what was directly taught in the classroom was how differently the same dataset needs to be organized depending on the question being asked of it — a hash table optimized for existence/frequency checks, a tree optimized for order, and a heap optimized for extremal access all had to stay consistent as events streamed in, which is a very different challenge from implementing any one structure in isolation. Implementing and rebalancing an AVL tree by hand also made concrete why self-balancing matters: an unbalanced BST under correlated insert order can silently degrade toward a linked list, destroying the O(log n) guarantee the whole design depends on. Running the empirical benchmark turned the abstract idea of "average-case O(1)" into something I could actually see change as load factor and data skew moved — it wasn't just a formula anymore.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press. (Hashing, AVL/balanced trees, and heap chapters.)
- Sedgewick, R., & Wayne, K. Algorithms. Addison-Wesley. (Balanced search trees and hash table collision resolution.)
- United Nations. The 17 Sustainable Development Goals — SDG 9: Industry, Innovation and Infrastructure; SDG 16: Peace, Justice and Strong Institutions. sdgs.un.org
- Course materials and rubric — CSA0309, Department of Computer Science and Engineering.
- AI-assisted tool: Claude (Anthropic) was used to help draft pseudo code, structure this report, and scaffold the visualization dashboard, under the student's direction and review, in accordance with the assignment format's requirement to acknowledge AI-assisted tools used.
- Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, Fundamentals of Data Structures in C,
- U niversities Press.
- 15. Rober t Lafore, Data Structures and Algorithms in C, Pearson.
- 16. GCC Documentation, GN U Compiler Collection.
- GitHub Documentation, Version Control and Repository Management. Course materials on Data Structures, Hashing, AV L Trees and Priority Queues.

6. Common Assessment Rubric (generic format)

Assessment Criterion	**Marks**
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

7. CO–PO–Assessment Mapping

Assessment Component	**CO**	**PO(s)**	**Bloom's Level**	**Marks**
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3, CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO3, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5

Documentation & reflection	CO5	PO10	L3/L4	5

8. Faculty Evaluation & Continuous Improvement — CO Attainment

Performance Level	**Criterion**
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____
Actual CO Attainment: _____
Faculty Analysis: Areas in which students performed well: _____
Areas requiring improvement:
_____
Corrective / Improvement Action:
_____
Action to be implemented in the next offering: _____